

AI Call Agent System

Well-Defined AI Build Prompt

For Use with Claude, GPT-4, or Any Advanced LLM

Version:	1.0
Date:	October 2026
Status:	Final
Purpose:	Recreate or extend the AI Call Agent system

Use this prompt in a new AI coding session to rebuild, extend, or debug the system.

Contents

1	How to Use This Document	2
2	Master Build Prompt	2
3	Component-Specific Prompts	4
3.1	AI Pipeline Only	4
3.2	Telephony Webhook Handler Only	5
3.3	Scheduler Module Only	6
3.4	Staff API Only	7
4	Debugging Prompt	8
5	Prompt Usage Tips	8
5.1	Best Practices When Using These Prompts	8
5.2	Follow-Up Prompts After Initial Build	9
6	Quick Reference Card	9

How to Use This Document

This document contains three prompts:

1. **Master Build Prompt** – Use this to build the entire system from scratch in a new AI session. It is comprehensive and self-contained.
2. **Component-Specific Prompts** – Use these when you want to build or fix one module (e.g., “I only need to fix the AI pipeline”).
3. **Debugging Prompt** – Use this when the system is running but something is broken.

Instructions: Copy the prompt text exactly. Paste it as your first message to the AI. Attach any relevant files (e.g., your current broken code) if using the debugging prompt.

Master Build Prompt

PASTE THIS ENTIRE TEXT AS YOUR FIRST MESSAGE TO THE AI:

MASTER BUILD PROMPT – Copy Everything Below This Line

You are a senior software architect and backend engineer. Build me a complete, production-ready AI Call Agent system for a college administration department. This system makes outbound phone calls to students, has a natural AI conversation with them in English, Hindi, or Gujarati, and logs transcripts for staff to review.

=== SYSTEM OVERVIEW ===

The system has 5 components:

1. Telephony Module (makes/receives calls via Plivo API)
2. AI Pipeline (STT -> LLM -> TTS, all local with cloud fallback)
3. Scheduler (Celery Beat dispatches calls at scheduled times)
4. Database Layer (PostgreSQL for all state, Redis for cache/queue)
5. Staff Web API (FastAPI REST endpoints + WebSocket for live monitoring)

=== HARDWARE ENVIRONMENT ===

Server: Ubuntu 22.04 LTS, single machine
GPU: NVIDIA with 24 GB VRAM (RTX 3090 or 4090)
RAM: 32-64 GB
Deployment: Docker + Docker Compose

=== TELEPHONY: PLIVO ===

Provider: Plivo (plivo.com)
Cost: Rs. 0.38/min outbound India calls
Auth: PLIVO_AUTH_ID and PLIVO_AUTH_TOKEN (from environment variables)
Caller ID: A Plivo DID number (India +91 number)
Call flow: Plivo calls student -> student answers -> Plivo sends webhook to our server -> our server returns XML with instructions -> dialogue loop

Plivo webhooks we must handle:

- POST /webhook/plivo/answer : when student answers; return XML
- POST /webhook/plivo/input : when student speaks; return XML with audio
- POST /webhook/plivo/hangup : when call ends; save transcript

Use the plivo Python SDK: `pip install plivo`

IMPORTANT: The adapter pattern is required. Create an abstract class `TelephonyAdapter` with methods: `place_call()`, `hangup_call()`, `get_call_status()`. `PlivoAdapter` implements this. This allows switching to Asterisk or Twilio later by only changing the adapter file.

=== AI MODELS (all local, with cloud fallback) ===

STT (Speech-to-Text):

Primary: IndicConformer (0.60B parameters, via NeMo toolkit)

- Model: ai4bharat/indicconformer
- Handles: Hindi, Gujarati, English
- Latency target: < 1.5 seconds

Fallback: Sarvam AI API (<https://api.sarvam.ai/speech-to-text>)

- Free tier: 100 min/day

Component-Specific Prompts

Use these when you only need to build or fix one part of the system.

AI Pipeline Only

Build the AI pipeline module for an outbound call agent system.

The pipeline has 3 stages: STT -> LLM -> TTS.

Each stage has a primary (local) model and a cloud fallback.

STT: IndicConformer (NeMo) primary, Sarvam AI API fallback

LLM: Qwen3-14B via Ollama primary, Groq API fallback

TTS: IndicF5 primary, gTTS fallback

Requirements:

1. Each stage is an abstract class (STTAdapter, LLMAdapter, TTSAdapter)
2. FallbackSTT/FallbackLLM/FallbackTTS wrap primary + fallback with `asyncio.wait_for()` timeout (STT: 2s, LLM: 5s, TTS: 3s)
3. `AIPipeline.process_turn(audio_url, context) -> audio_path`
 - Total latency target: < 4 seconds
 - Log each stage's latency to `context["latency_log"]`
4. All methods are `async`
5. `context` dict has: `system_prompt`, `history` (last 6 msgs), `lang`, `turn`

Environment variables:

- `SARVAM_API_KEY`
- `GROQ_API_KEY`
- `OLLAMA_BASE_URL` (default: `http://localhost:11434`)

Write the files: `stt.py`, `llm.py`, `tts.py`, `pipeline.py`

Include error handling and logging.

Telephony Webhook Handler Only

Build the Plivo webhook handler for an AI call agent.

The system uses Plivo for outbound calling. When a student answers, Plivo sends webhook POST requests to our FastAPI server.

Implement these 3 webhook endpoints as a FastAPI APIRouter:

POST /webhook/plivo/answer

- Receives: CallUUID, From, To in POST body
- Action: Load call context from PostgreSQL; store in Redis;
run TTS on opening message; return Plivo XML with Play + Record

POST /webhook/plivo/input

- Receives: RecordingUrl, CallUUID in POST body
- Action: Download audio; run AI pipeline (STT->LLM->TTS);
update Redis context; return Plivo XML with new audio

POST /webhook/plivo/hangup

- Receives: CallUUID, Duration in POST body
- Action: Update call_tasks record; write transcript to call_logs;
broadcast WebSocket event

Plivo XML format example:

```
<Response>
  <Play>https://example.com/audio/response.wav</Play>
  <Record action="/webhook/plivo/input?task_id=123"
    maxLength="30" silence="3" finishOnKey="#" />
</Response>
```

For hangup after N turns:

```
<Response>
  <Speak>Thank you for your time. Goodbye!</Speak>
  <Hangup/>
</Response>
```

Database models available (SQLAlchemy):

- CallTask: id, campaign_id, student_id, plivo_uuid, status, script_id
- CallLog: id, task_id, transcript (JSONB), stt_provider, llm_provider

Redis keys:

- call:session:{call_uuid} -> hash with: system_prompt, history JSON,
lang, turn_count, task_id

Include: error handling, logging, IP validation for Plivo ranges

Scheduler Module Only

Build the Celery scheduler for an outbound call campaign system.

Technology: Celery 5.x with Redis broker, Celery Beat for periodic tasks

Tasks to implement:

1. `dispatch_due_calls()` - Celery Beat task, runs every 30 seconds
 - Query PostgreSQL for `call_tasks` WHERE `status='pending'` AND `scheduled_at <= NOW()`
 - For each task: call `place_call_task.delay(task.id)`
 - Limit: max 10 concurrent dispatches per run (to avoid burst)
2. `place_call_task(call_task_id)` - Celery task
 - `max_retries=3`, `default_retry_delay=300` (5 minutes)
 - Load `CallTask` from DB
 - Call `PlivoAdapter.place_call()` with `answer_url` and `hangup_url`
 - Update `task.status`, `task.plivo_uuid`, `task.placed_at`
 - On failure: `self.retry(exc=...)`
 - On final failure (retries exhausted): set `status='failed'`
3. `retry_no_answer_calls()` - Celery Beat task, runs every 15 minutes
 - Find tasks with `status='no_answer'` AND `retry_count < campaign.max_retries`
 - Reschedule with delay of `campaign.retry_delay_min` minutes

Celery Beat schedule:

```
dispatch_due_calls: every 30 seconds
retry_no_answer_calls: every 15 minutes
cleanup_old_sessions: daily at 2 AM
```

Environment:

```
REDIS_URL=redis://localhost:6379/0
DATABASE_URL=postgresql+asyncpg://user:pass@localhost:5432/ai_calls
BASE_URL=https://calls.college.edu
PLIVO_AUTH_ID, PLIVO_AUTH_TOKEN, PLIVO_CALLER_ID
```

Write: `tasks.py` with full error handling and structured logging.

Staff API Only

Build the FastAPI REST API for staff to manage the AI call agent system.

Authentication: JWT with RS256, 24h access token, 7d refresh token

Library: python-jose[cryptography], passlib[bcrypt]

Roles: admin (full access), operator (read + trigger only)

Endpoints:

POST /auth/login

Body: {"email": "...", "password": "..."}
Returns: {"access_token": "...", "refresh_token": "...", "role": "..."}
}

POST /auth/refresh

Body: {"refresh_token": "..."}
Returns: {"access_token": "..."}
}

GET /campaigns?status=pending&page=1&limit=20

Returns: paginated list with counts per status

POST /campaigns [admin only]

Body: {name, script_id, target_filter, scheduled_at, max_retries}
Action: Create campaign, generate call_tasks for matching students

GET /campaigns/{id}

Returns: campaign details + task counts by status

PATCH /campaigns/{id}/cancel [admin only]

Action: Set campaign status='cancelled', cancel pending tasks

GET /calls/{task_id}

Returns: task status, timing, outcome

GET /calls/{task_id}/transcript

Returns: full call transcript from call_logs

POST /calls/{task_id}/retry [admin only]

Action: Reset task to pending, trigger immediately

GET /analytics/daily?days=7

Returns: daily call volume, success rate, avg duration per day

GET /analytics/latency

Returns: p50, p95, p99 for STT, LLM, TTS from last 24h

GET /analytics/costs?days=30

Returns: daily spend (sum(duration_sec) * Rs.0.38 / 60) per day

WebSocket /ws/calls?token={jwt} 7

On connect: verify JWT

Events broadcast: {type, task_id, timestamp, data}

Debugging Prompt

You are debugging an AI Call Agent system. Here is the system summary:

SYSTEM: Outbound call agent for a college. Makes calls via Plivo, uses Qwen3-14B LLM (Ollama), IndicConformer STT (NeMo), IndicF5 TTS. Backend: FastAPI + Celery + PostgreSQL + Redis. Ubuntu 22.04 + Docker.

[ATTACH YOUR BROKEN CODE FILES OR PASTE ERROR LOGS BELOW]

For each bug you find:

1. State exactly which file and line has the problem
2. Explain why it's wrong (root cause, not just symptom)
3. Show the corrected code snippet
4. State if this change affects any other file

Also check for these common issues in this system:

- Async/sync mixing (Plivo SDK is sync; use `run_in_executor` for it)
- Plivo XML formatting errors (use `xml.etree.ElementTree`, not f-strings)
- Redis key TTL not set (all `call:session` keys must have `TTL=600`)
- SQLAlchemy async session not closed after webhook handler
- IndicConformer running on CPU instead of GPU (check `CUDA_VISIBLE_DEVICES`)
- Ollama not finding the `qwen3:14b` model (must be pulled first)
- Celery Beat not running (separate container/process from worker)
- JWT secret not set (causes 500 on all auth endpoints)

[PASTE YOUR ERROR MESSAGE OR BROKEN CODE HERE]

Prompt Usage Tips

Best Practices When Using These Prompts

1. **Start a fresh session:** These prompts work best in a new AI session with no prior context.
2. **Paste exactly:** Do not summarize or shorten the master prompt. The specificity is intentional. Every sentence removes ambiguity.
3. **Request one component at a time:** If the full system feels too large, use Section 3 component prompts. Build in this order:
 - `config.py` and `database/models.py` first
 - AI pipeline second (can be tested standalone)
 - Telephony adapter third
 - Scheduler fourth
 - API last

4. **Verify the AI's output:** After the AI generates code, ask: "What tests would you write for this? Write them." Then: "Run the tests mentally and tell me if any would fail."
5. **For the webhook handler:** The AI must know your Plivo credentials are in environment variables, and that webhooks must be reachable from the internet. If testing locally, tell the AI you are using ngrok.
6. **For Ollama:** Remind the AI: "Ollama is running in a separate Docker container at `http://ollama:11434`. The model `qwen3:14b` is already pulled."

Follow-Up Prompts After Initial Build

After the AI generates the initial code, use these follow-up prompts:

Follow-Up 1: Add Monitoring

Add Prometheus metrics instrumentation to this FastAPI app.

Track these metrics:

- `ai_pipeline_latency_seconds` (histogram, labels: `stage=stt/llm/tts`, `provider=primary/fallback`)
 - `plivo_calls_total` (counter, labels: `outcome=completed/failed/no_answer`)
 - `active_calls_current` (gauge)
 - `celery_tasks_total` (counter, labels: `task_name`, `status`)
- Expose `/metrics` endpoint. Use `prometheus_client` library.

Follow-Up 2: Add WhatsApp Fallback

Add WhatsApp message fallback for students who don't answer calls.

When a call has status `'no_answer'` after `max_retries`, send a WhatsApp message via Plivo's WhatsApp API summarizing the call's purpose.

Use the same campaign script's `opening_message` as the WhatsApp text.

Add a new field: `students.whatsapp_number` (nullable VARCHAR 15).

Add a Celery task: `send_whatsapp_followup(task_id)` triggered after final `no_answer` retry.

Follow-Up 3: Student Import

Add a CSV import endpoint for bulk student upload.

POST `/admin/students/import` - multipart form with CSV file

CSV columns: `roll_number`, `name`, `phone`, `department`, `semester`, `language_pref`

Validate: phone must be 10 digits; `language_pref` in `[hi, gu, en]`

Return: `{"imported": 50, "skipped": 3, "errors": [{"row": 5, "reason": "..."}]}`

Use pandas for CSV parsing. Max 1000 rows per upload.

Quick Reference Card

Item	Value
Telephony provider	Plivo
Outbound call cost (India)	Rs. 0.38/min
DID number cost	Rs. 200/month
STT model (local)	IndicConformer 0.60B
STT API fallback	Sarvam AI (free 100 min/day)
LLM model (local)	Qwen3-14B via Ollama
LLM API fallback	Groq llama3-70b (14,400 req/day free)
TTS model (local)	IndicF5 0.40B
TTS fallback	gTTS (Google, always free)
Languages	Hindi (hi), Gujarati (gu), English (en)
Latency target	< 4s per conversation turn
Capacity	200-300 calls/day on single server
Database	PostgreSQL 15
Cache/Queue	Redis 7
Task queue	Celery 5 + Celery Beat
API framework	FastAPI (Python 3.11)
Container	Docker + Docker Compose
Reverse proxy	Nginx (TLS termination)
Monitoring	Prometheus + Grafana

Table 1: Quick Reference

AI Build Prompt – AI Call Agent v1.0

Keep this document with the system design and SRS for future reference.